

EE216 Reconfigurable Computing

Topic 11: Programming for SDSoC

Prof. Yajun Ha

ShanghaiTech University
Shanghai, China



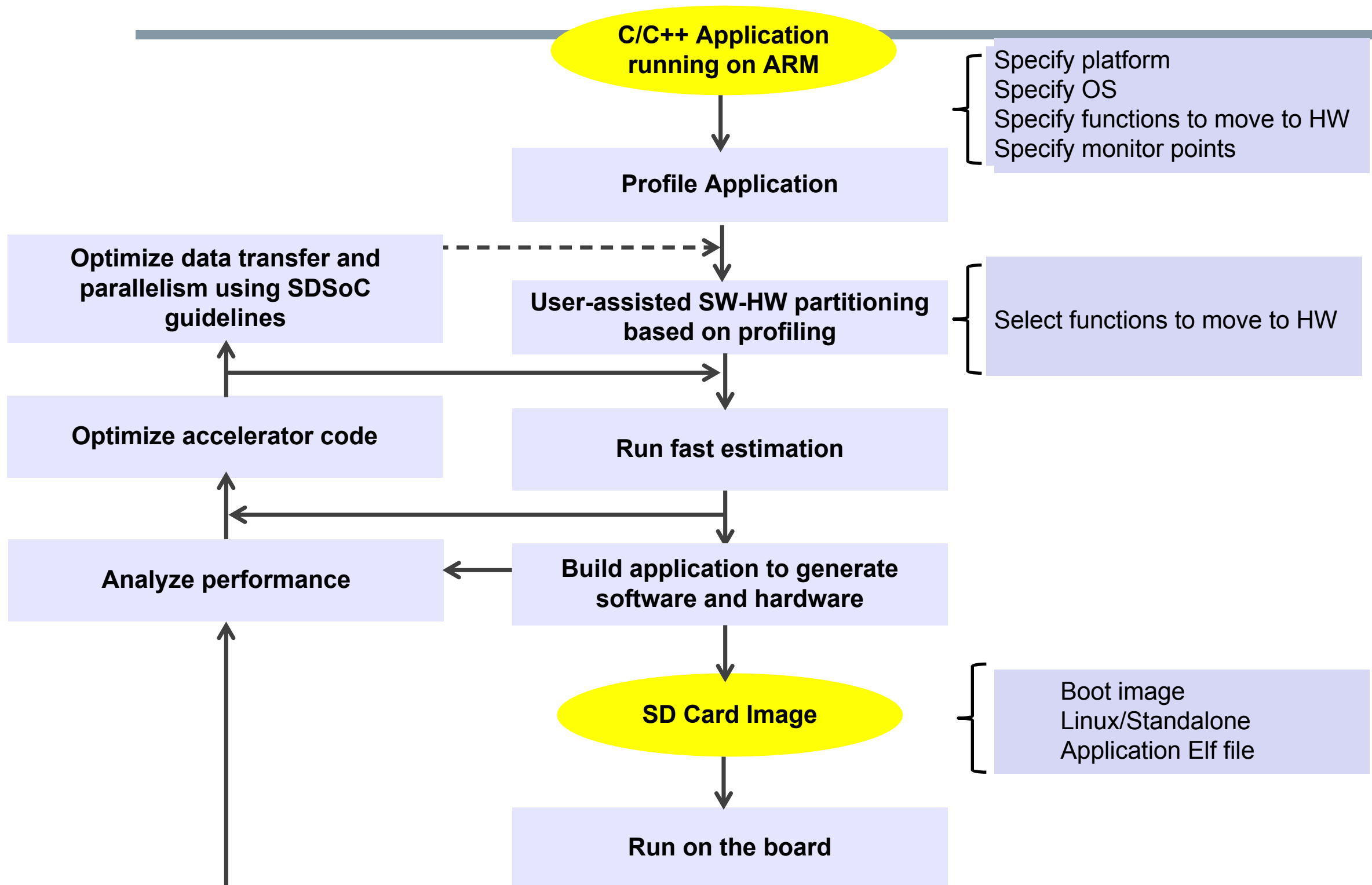
上海科技大学
ShanghaiTech University

Programming for SDSoC

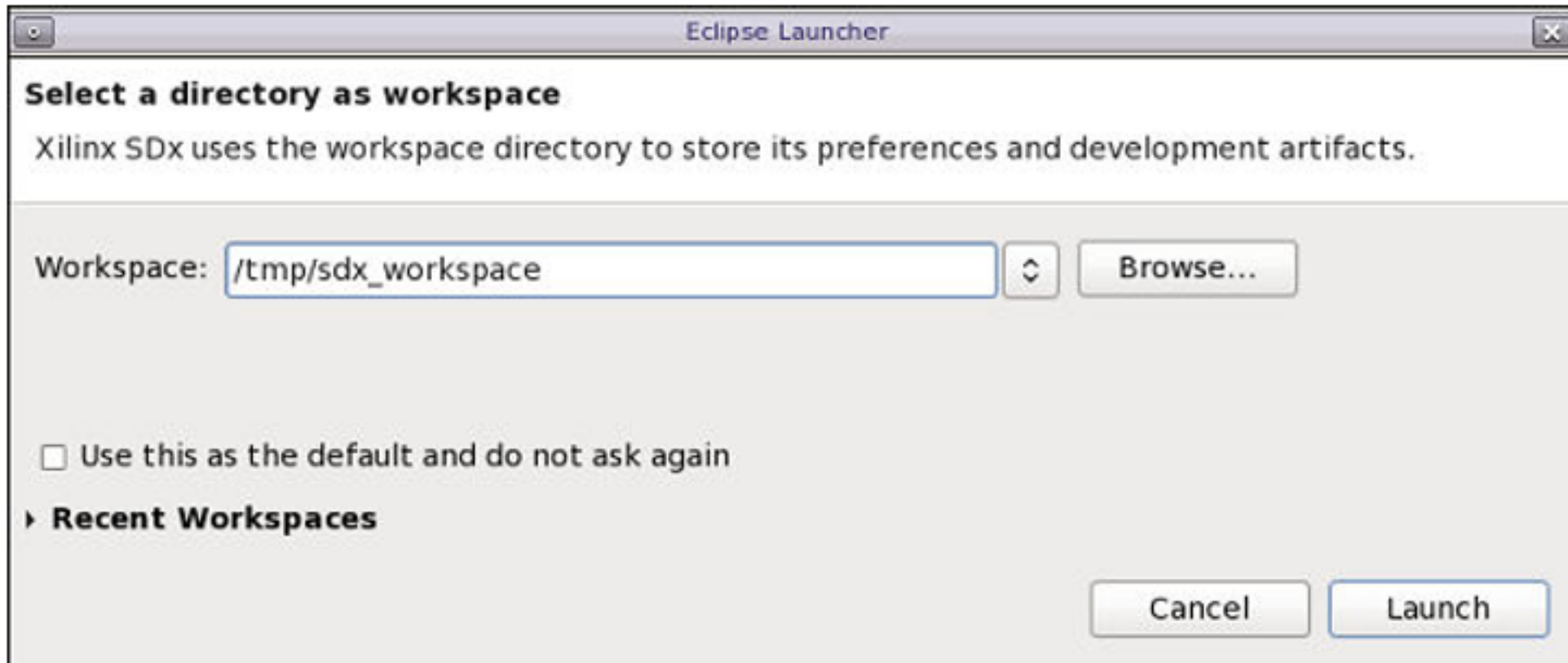
- 👉 Create, Build and Run a SDSoC Project
- SDSoC Execution and Building
- Programming for SDSoC



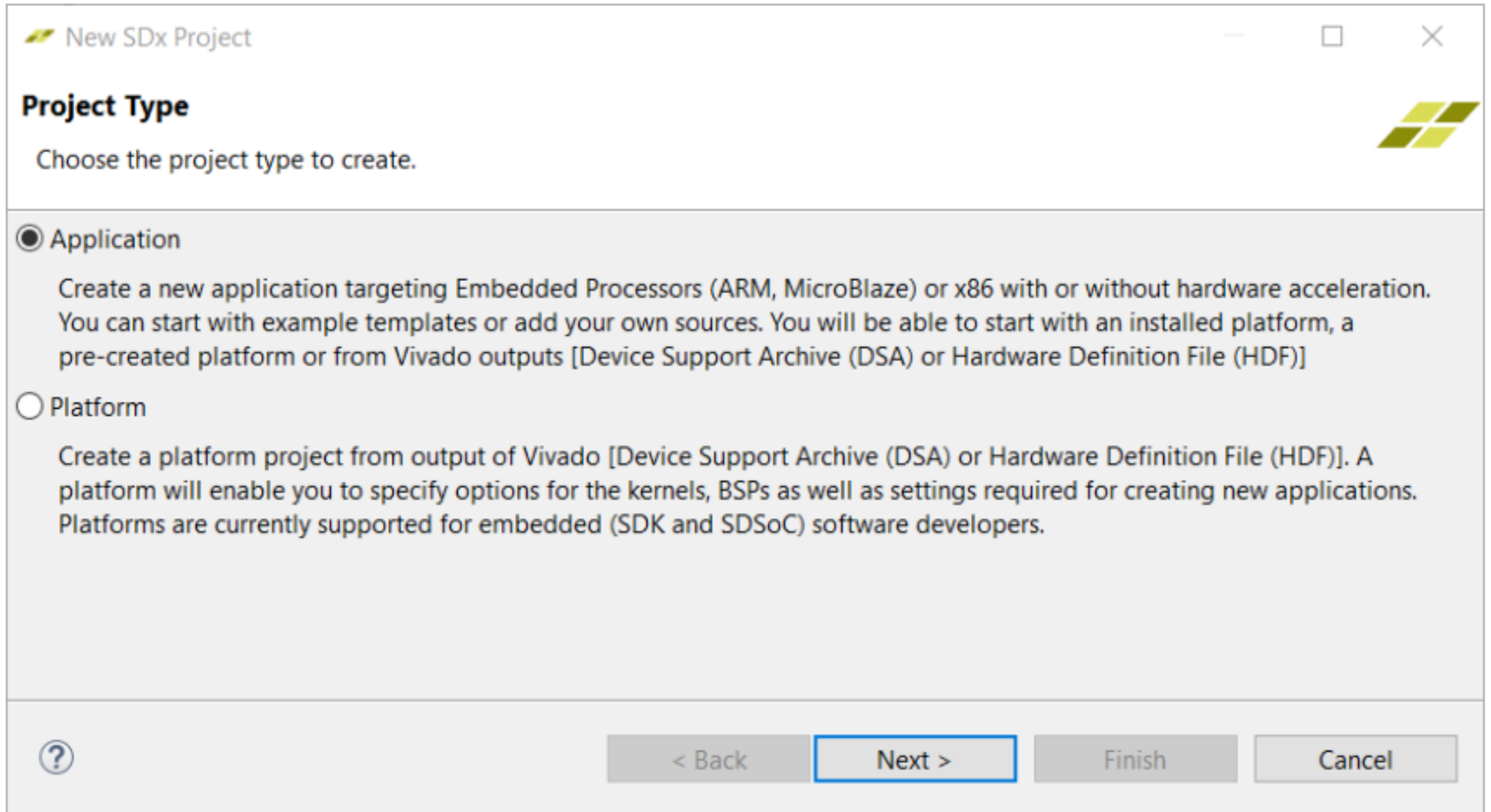
Development Flow



Step 1: Using an SDx Workspace



Step 1: Selecting Project Types



New SDx Project

Project Type

Choose the project type to create.

☒ Application

Create a new application targeting Embedded Processors (ARM, MicroBlaze) or x86 with or without hardware acceleration. You can start with example templates or add your own sources. You will be able to start with an installed platform, a pre-created platform or from Vivado outputs [Device Support Archive (DSA) or Hardware Definition File (HDF)]

☐ Platform

Create a platform project from output of Vivado [Device Support Archive (DSA) or Hardware Definition File (HDF)]. A platform will enable you to specify options for the kernels, BSPs as well as settings required for creating new applications. Platforms are currently supported for embedded (SDK and SDSoc) software developers.

? < Back Next > Finish Cancel



Step:1 Selecting a Platform

New SDx Project

Platform

Choose a platform for your project

Type: ☒ Platform ☐ Hardware specification (DSA/HDF)

Platforms (6) [Filter](#)

Find:

Name	Board	Family	Part	Version	Vendor	Flow	Location
zc702	zc702	zynq	xc7z020	1.0	xilinx.com	SDSoC	\$XILINX_SDX\platforms\zc702\zc702.xpfm
zc706	zc706	zynq	xc7z045	1.0	xilinx.com	SDSoC	\$XILINX_SDX\platforms\zc706\zc706.xpfm
zcu102	zcu102	zynqplus	xczu9eg	1.0	xilinx.com	SDSoC	\$XILINX_SDX\platforms\zcu102\zcu102.xpfm
zcu104	zcu104	zynqplus	xczu7ev	1.0	xilinx.com	SDSoC	\$XILINX_SDX\platforms\zcu104\zcu104.xpfm
zcu106	zcu106	zynqplus	xczu7ev	1.0	xilinx.com	SDSoC	\$XILINX_SDX\platforms\zcu106\zcu106.xpfm
zed	zed	zynq	xc7z020	1.0	xilinx.com	SDSoC	\$XILINX_SDX\platforms\zed\zed.xpfm

Add Custom Platform...

Manage Platform Repositories...

Add Devices/Platforms...

Description

Basic platform targeting the ZC702 board, which includes 1GB of DDR3, 16MB Quad-SPI Flash and an SDIO card interface. More information at <https://www.xilinx.com/products/boards-and-kits/ek-z7-zc702-g.html>

Repository: \$XILINX_SDX/platforms/zc702



Step 1: System Configuration

 New SDx Project

System configuration

Provide the system configuration and software details for your project

Software Platform

System configuration: Linux

Runtime: C/C++

Domain

Domain: linux

CPU: cortex-a9

OS: linux

Additional Settings

☐ Linux Root File System:

Output type

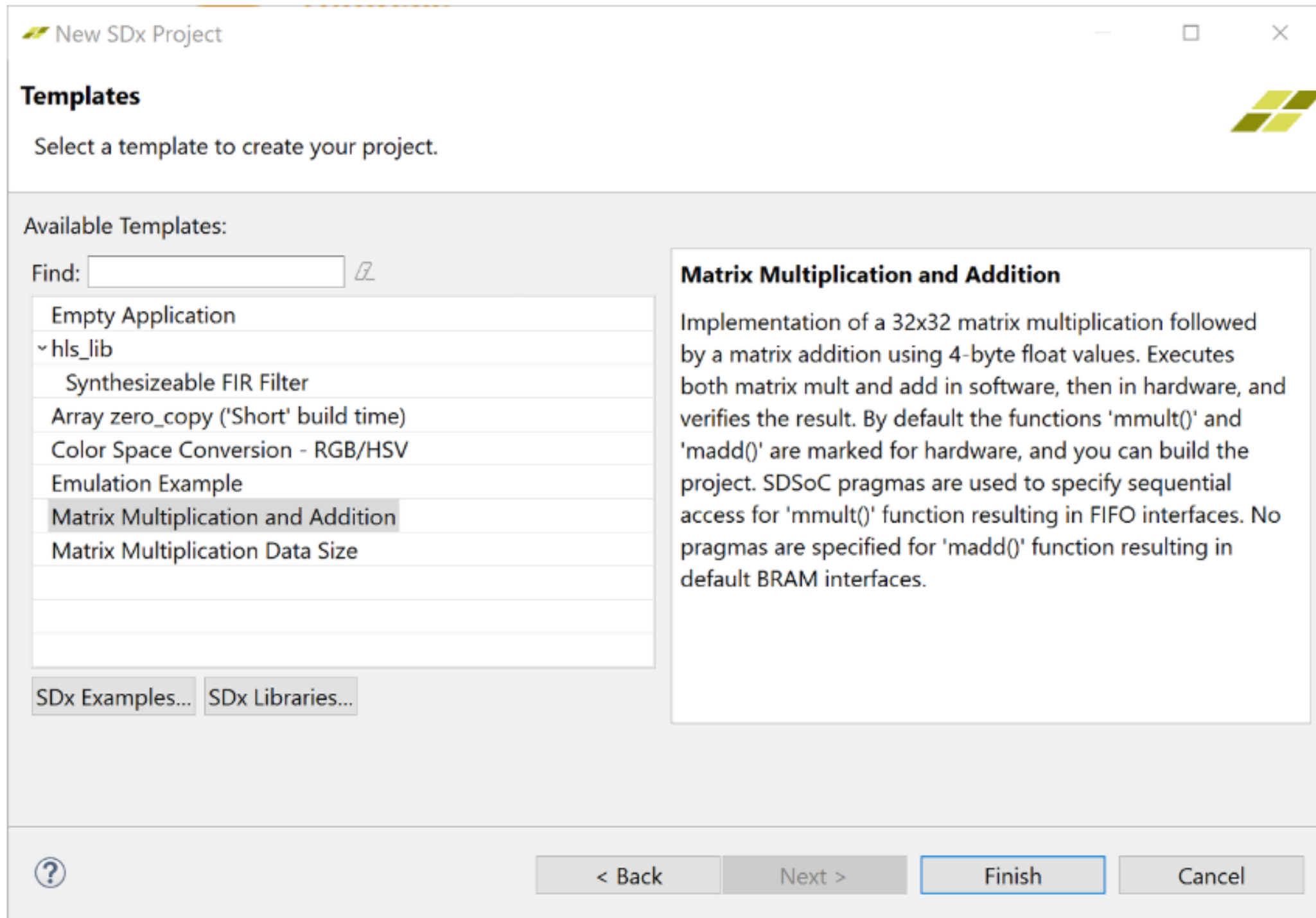
☒ Executable (elf) ☐ Shared Library ☐ Static Library ☐ C-Callable Library

☒ Allow hardware accelerations

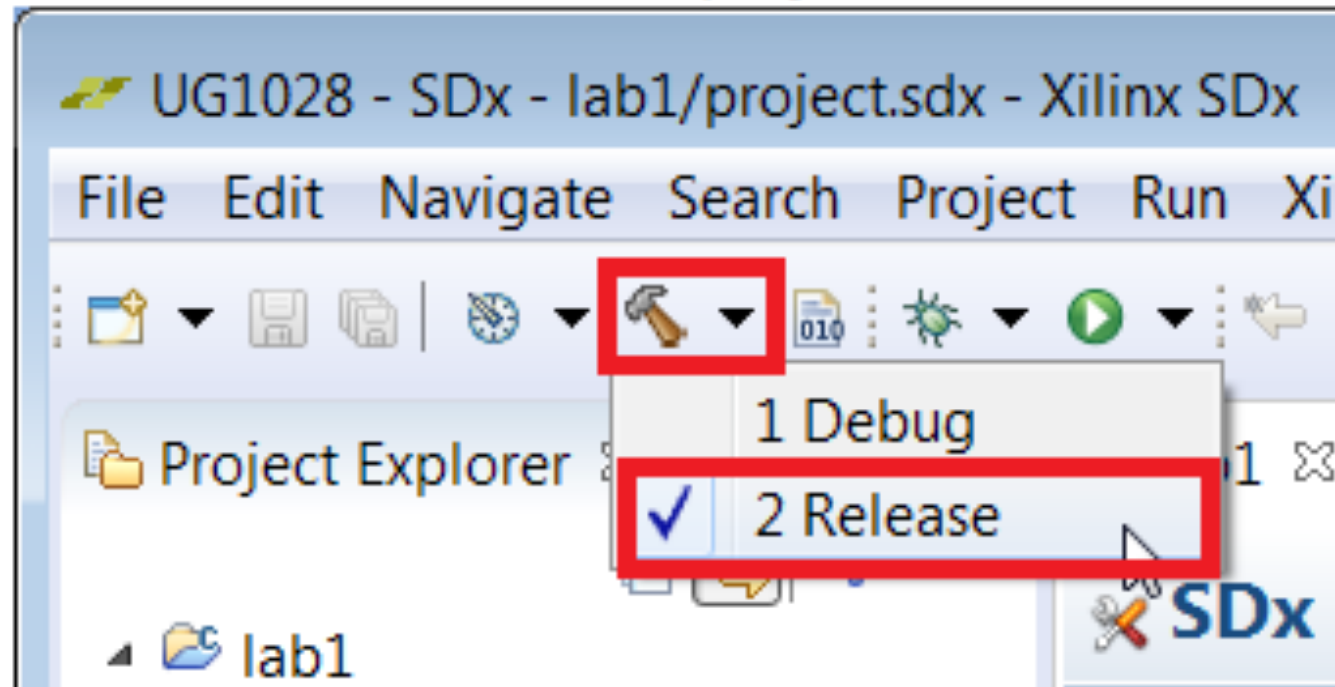




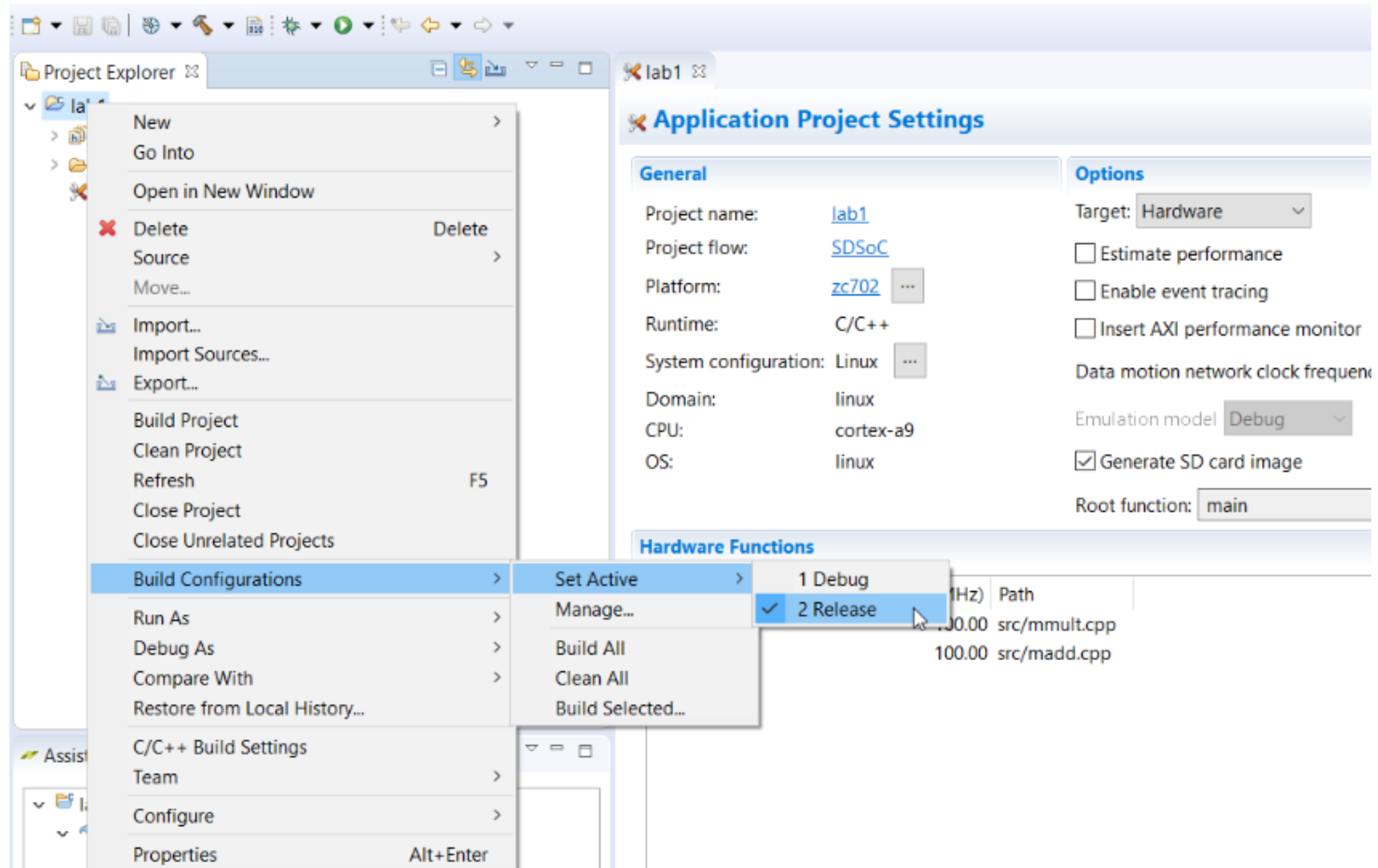
Step 1: Selecting a Template



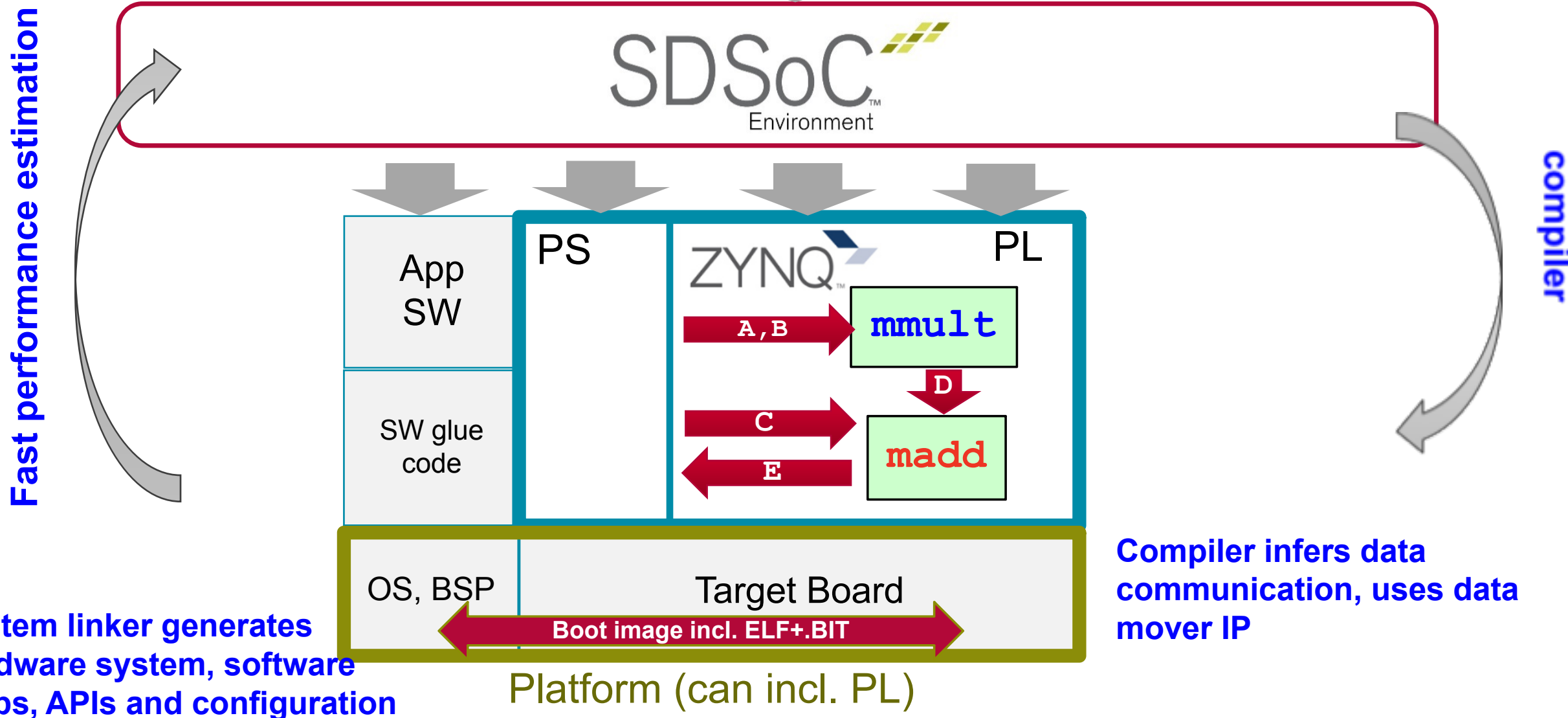
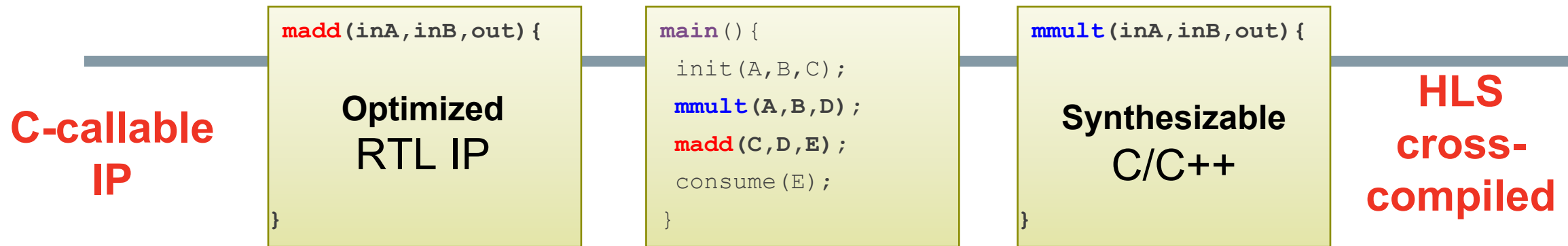
Step 1: Build Configuration



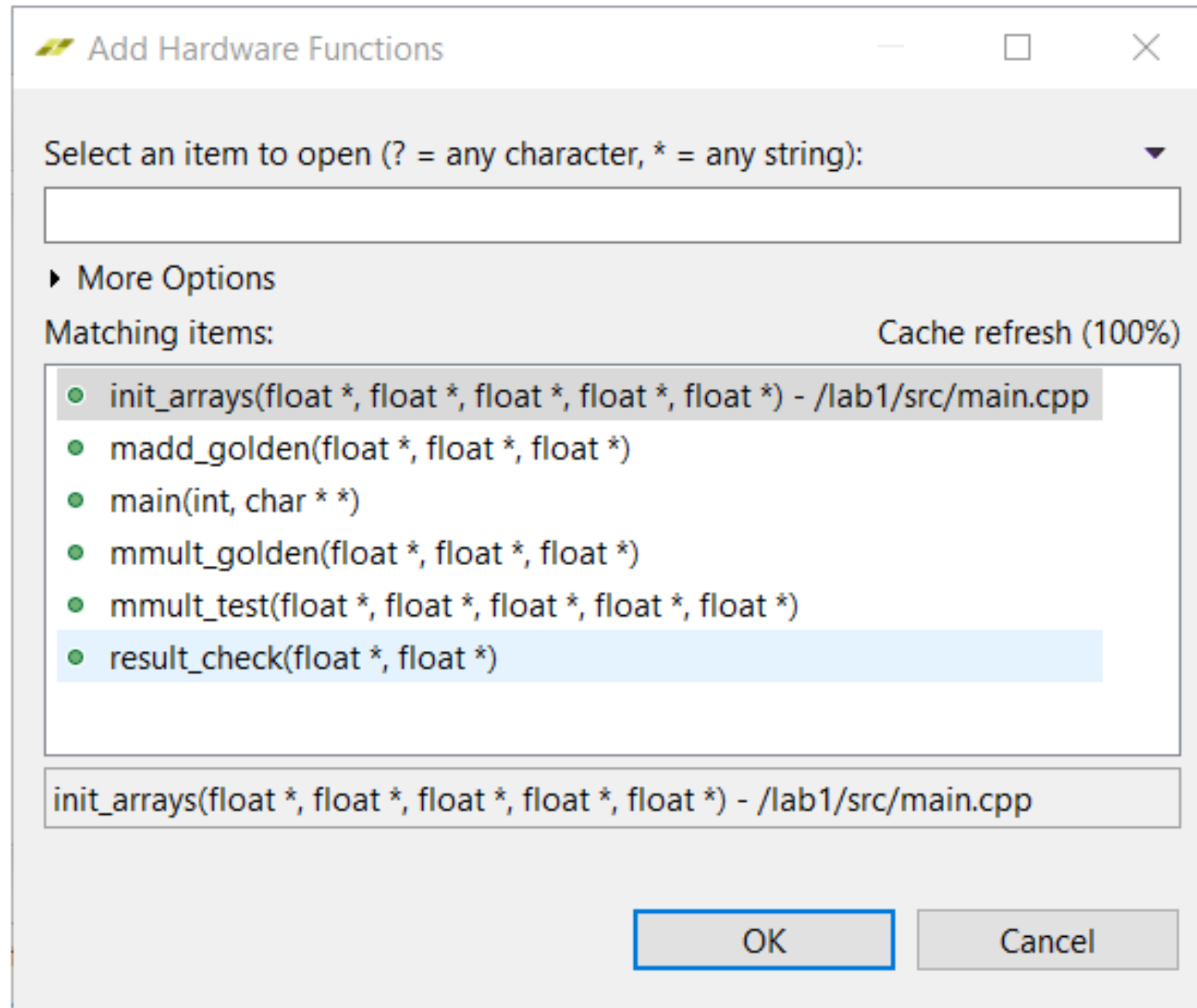
Step 1: Set Active Build Configuration



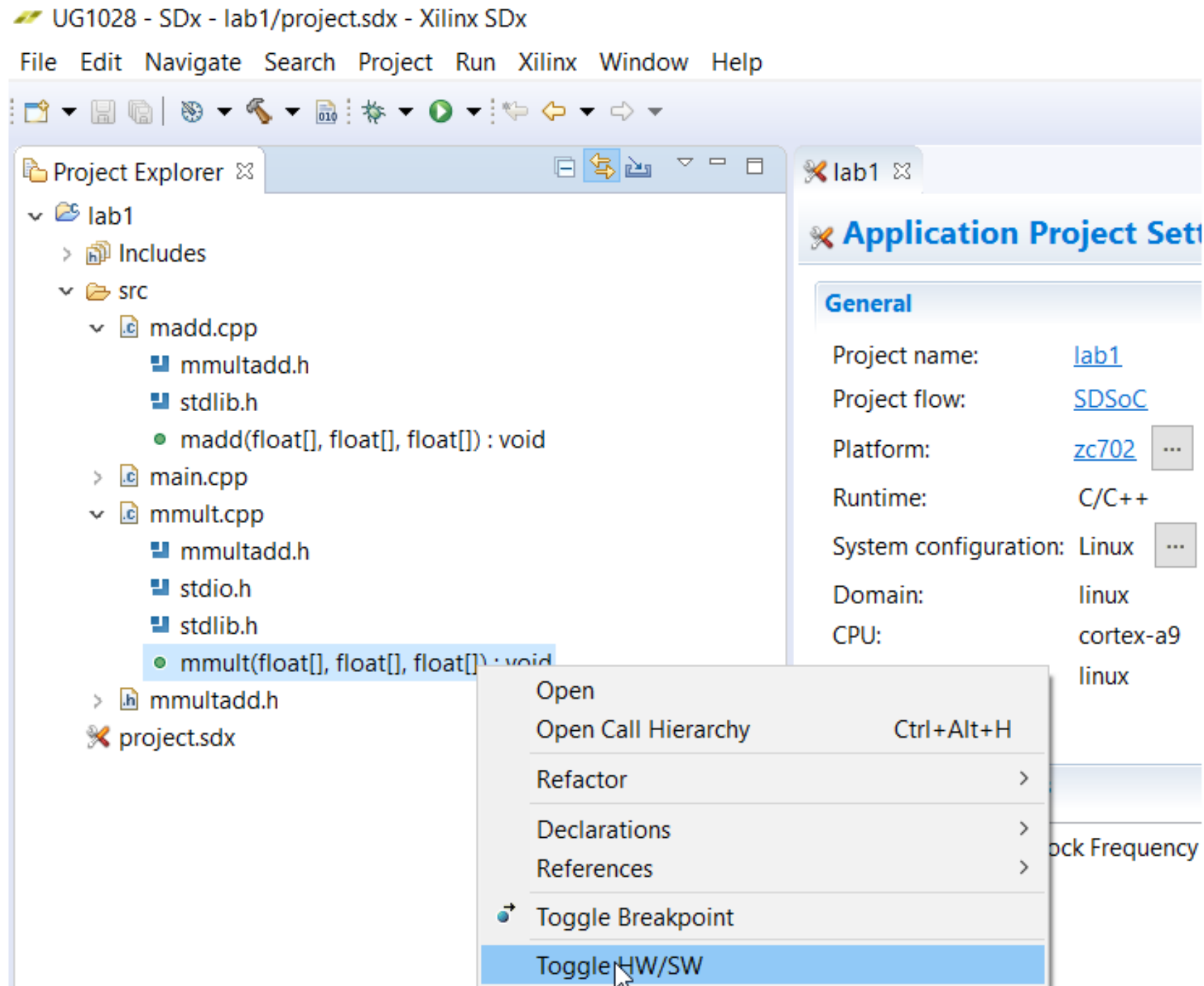
A SDSoC Design for an Algorithm



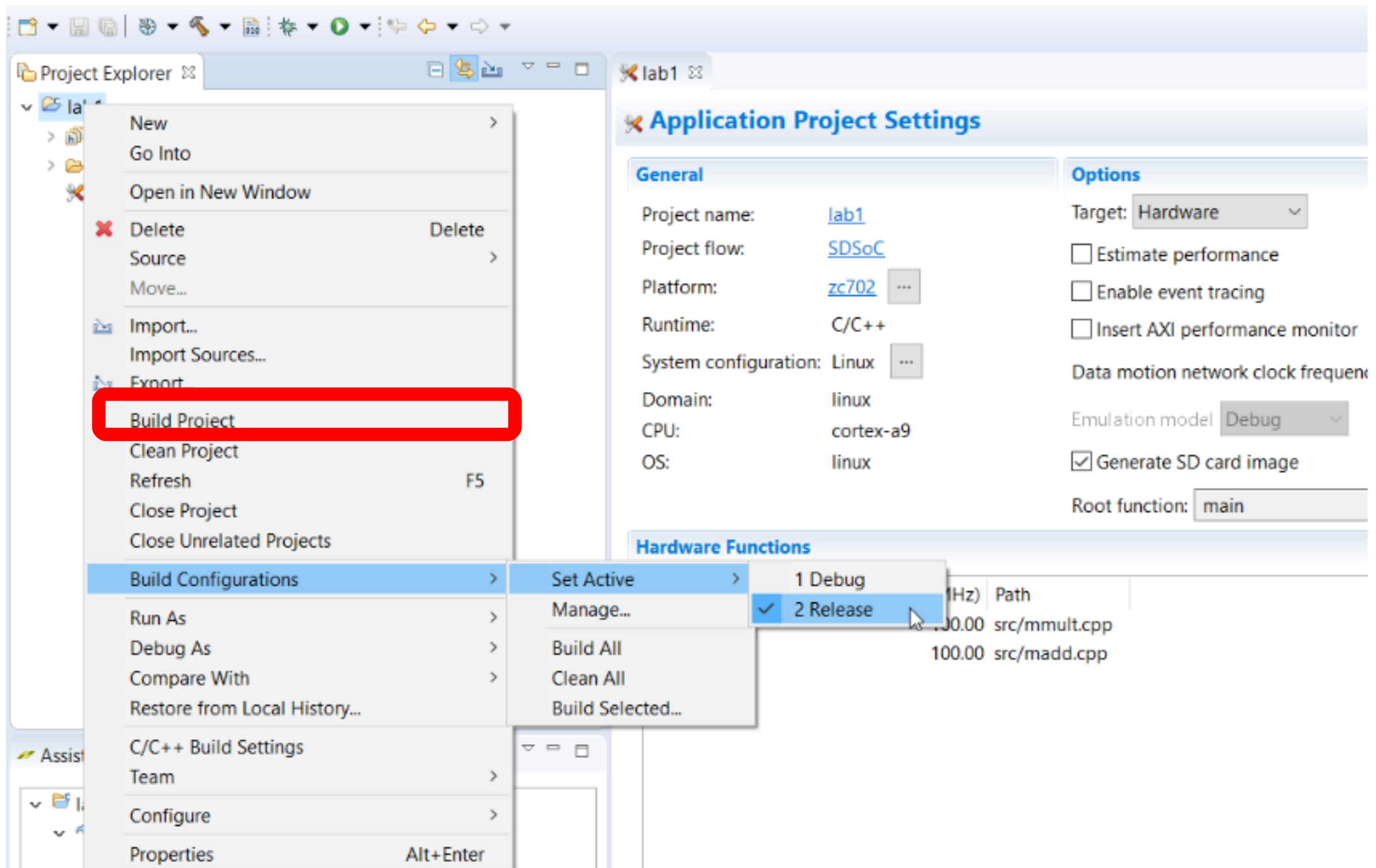
Step 2: Add Hardware Functions



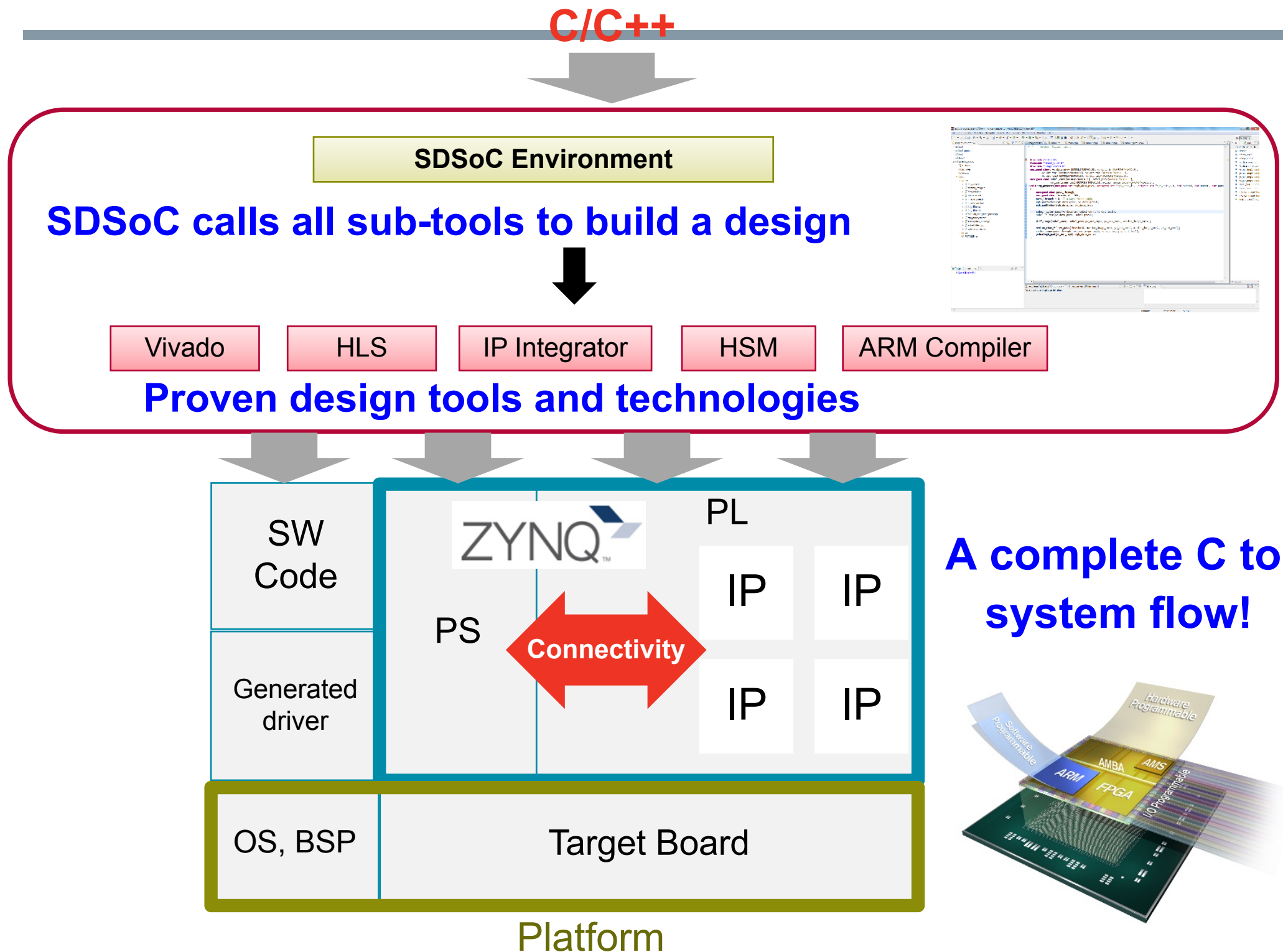
Step 2: Alternative Way to Add Hardware Functions



Step 3: Build a Design



Step 3: Build a Design



Building the Complete System

- Building a project is where SDSoC does all the heavy lifting

Three basic methods to run the build process

Through the IDE, using a makefile, directly from the command line

It is during this step that SDSoC calls upon its internal framework and several sub-executables to accomplish

(1) system compilation, (2) system analysis, (3) system generation

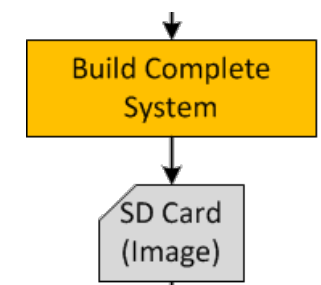
- Activate the Release build configuration for a fully optimized build
- A successful build will result in several output products including the following key items

Application program in ELF format

Bitstream (optional)

SD card image (optional)

Design projects (e.g. Vivado Design Suite and Vivado HLS tool)



Step 3: Data Motion Network

Data Motion Network

Accelerator	Argument	IP Port	Direction	Declared Size(bytes)	Pragmas	Connection
madd_1	A	A	IN	1024*4		mmult_1:C
	B	B	IN	1024*4		ps7_S_AXI_ACP:AXIDMA_SIMPLE
	C	C	OUT	1024*4		ps7_S_AXI_ACP:AXIDMA_SIMPLE
mmult_1	A	A	IN	1024*4		ps7_S_AXI_ACP:AXIDMA_SIMPLE
	B	B	IN	1024*4		ps7_S_AXI_ACP:AXIDMA_SIMPLE
	C	C	OUT	1024*4		madd_1:A



Step 3: Accelerator Callsite

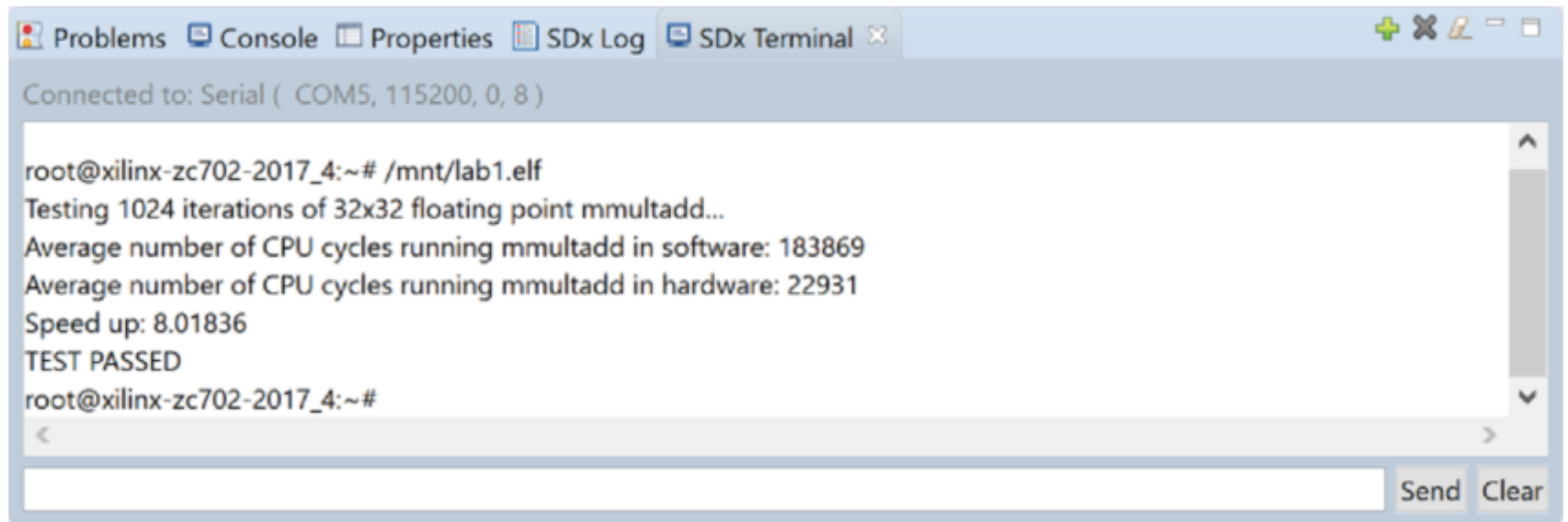
Accelerator Callsites

Accelerator	Callsite	IP Port	Transfer Size(bytes)	Paged or Contiguous	Datamover Setup Time(CPU cycles)	Transfer Time(CPU cycles)
madd_1	main.cpp:128:11	A	4096	paged		
		B	4096	contiguous	1112	7976
		C	4096	contiguous	1112	7976
mmult_1	main.cpp:127:11	A	4096	contiguous	1112	7976
		B	4096	contiguous	1112	7976
		C	4096	paged		

Open the lab1/Release/_sds/swstubs/mmult.cpp file, to see how the SDx system compiler replaced the original mmultfunction with one named _p0_mmult_1_noasync that performs transfers to and from the FPGA using cf_send_i and cf_wait functions. The SDx system compiler also replaces calls to mmult with _p0_mmult_1_noasync in lab1/Release/_sds/swstubs/main.cpp. The SDx system compiler uses these rewritten source files to build the ELF that accesses the hardware functions.



Step 4: Run the Project



The screenshot shows the SDx Terminal window with the following text:

```
Connected to: Serial ( COM5, 115200, 0, 8 )

root@xilinx-zc702-2017_4:~# /mnt/lab1.elf
Testing 1024 iterations of 32x32 floating point mmultadd...
Average number of CPU cycles running mmultadd in software: 183869
Average number of CPU cycles running mmultadd in hardware: 22931
Speed up: 8.01836
TEST PASSED
root@xilinx-zc702-2017_4:~#
```

The terminal window has tabs for Problems, Console, Properties, SDx Log, and SDx Terminal. The SDx Terminal tab is active. The output shows the execution of a test program that compares software and hardware execution times for a 32x32 floating point multiplication function (mmultadd). The hardware execution is significantly faster, resulting in a speedup of approximately 8 times.

The speedup is 8 times faster, when the function is accelerated in hardware. The application running on the processor takes about 184K cycles while the application running on both the processor and the FPGA takes about 22K cycles.



After completing this tutorial, you should be able to do the following:

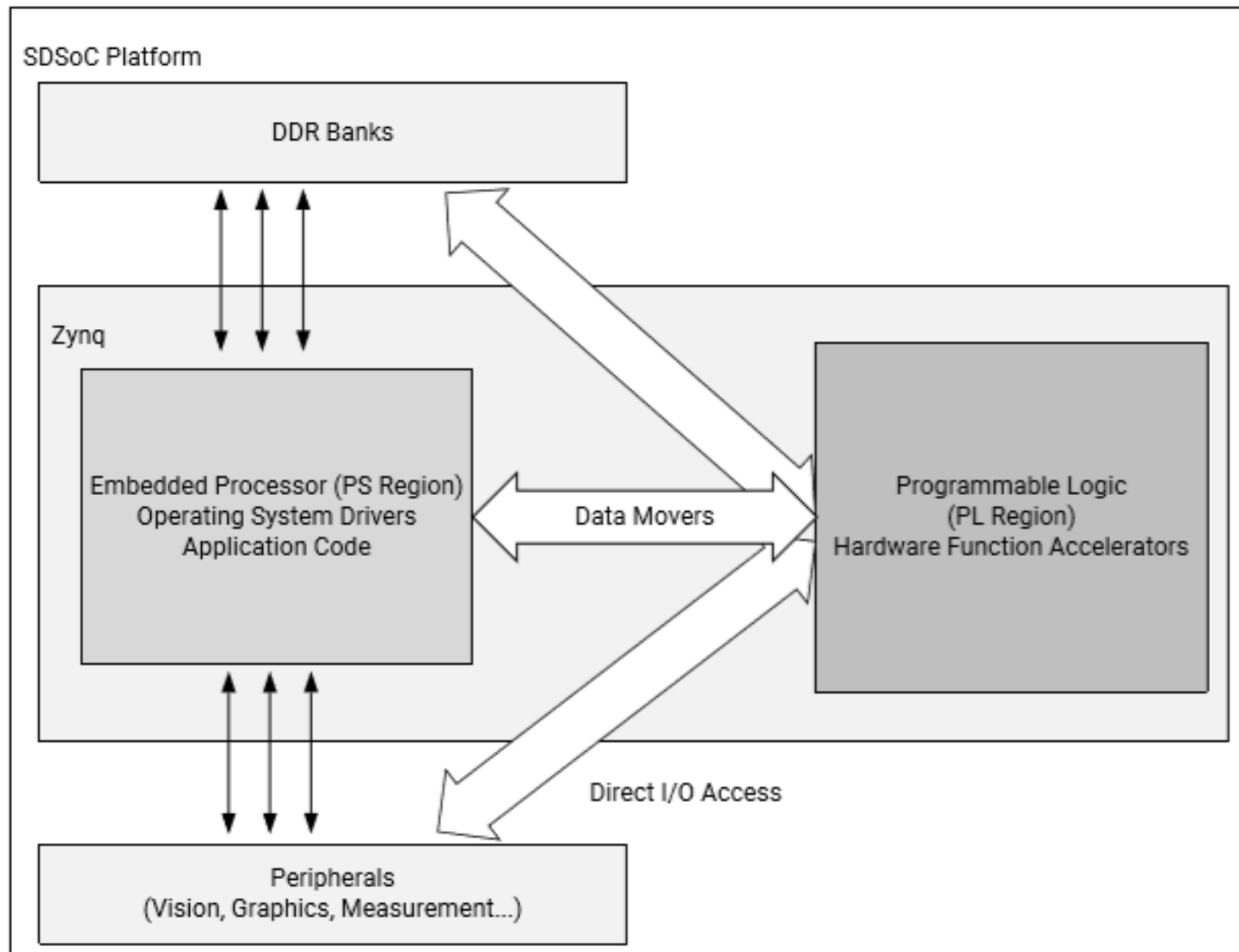
- Create a new SDSoC environment project for your application from a number of available platforms and project templates.
- Mark a function for hardware implementation.
- Build your project to generate a bitstream containing the hardware implemented function and an executable file that invokes this hardware implemented function.

Programming for SDSoC

- Create a SDSoC Project
- 👉 SDSoC Execution and Building
- Programming for SDSoC



Execution Model of an SDSoC Application



How C++ Executable Interfaces to Hardware

- Hardware functions within a program is compiled into hardware accelerators that are accessed with the standard C runtime through calls into these functions.
- Each hardware function call invokes the accelerator as a task and each of the arguments to the function is transferred between the CPU and the accelerator.
- Data transfers between memory and accelerators are accomplished through data movers, such as a DMA engine, automatically inserted into the system by the sds++ system compiler.

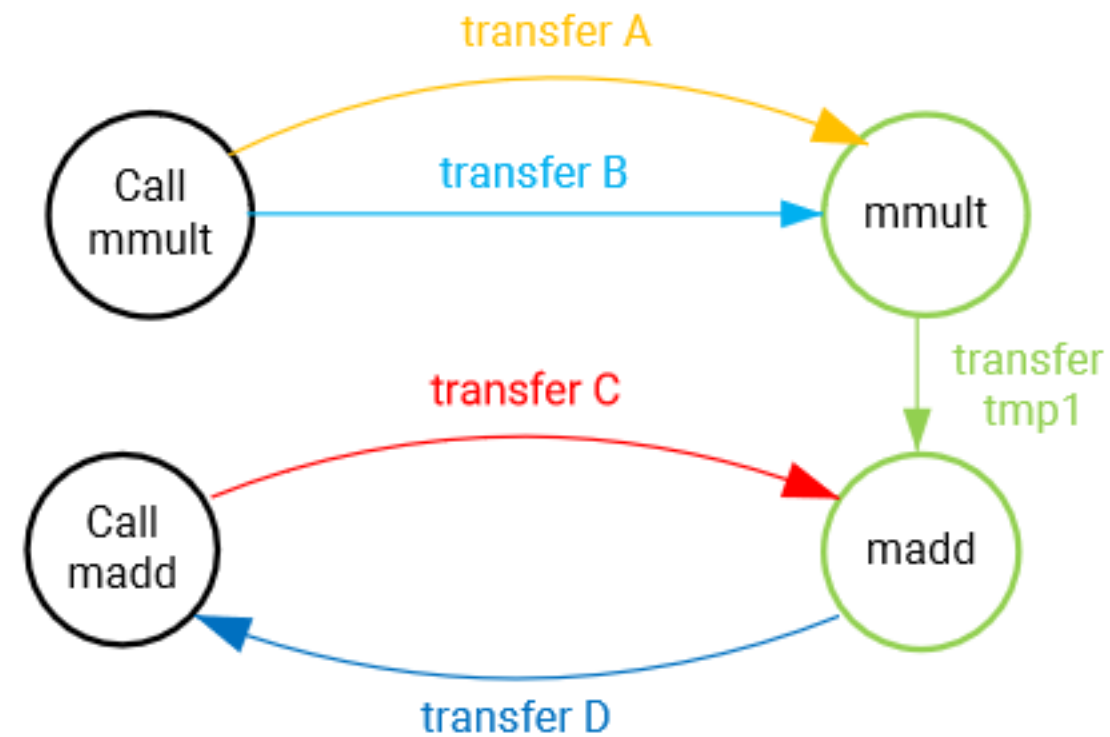
Hardware/Software Connectivity

```
bool mmultadd_test(float *A, float *B, float *C, float *Ds, float *D)
{
    float tmp1[A_NROWS * A_NCOLS], tmp2[A_NROWS * A_NCOLS];
    for (int I = 0; ii < NUM_TESTS; i++) {
        mmultadd_init(A, B, C, Ds, D);

        mmult(A, B, tmp1);
        //std::cout << "tmp1[0] = " << tmp1[0] << std::endl;
        madd(tmp1, C, D);

        mmult_golden(A, B, tmp2);
        madd_golden(tmp2, C, Ds);

        if (!mmult_result_check(D, Ds))
            return false;
    }
    return true;
}
```



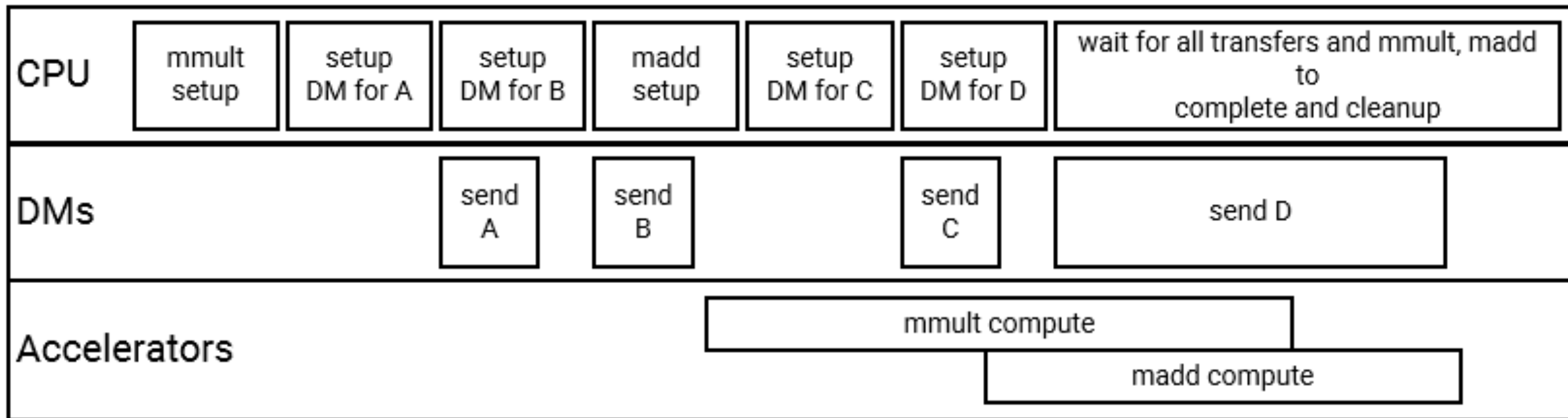
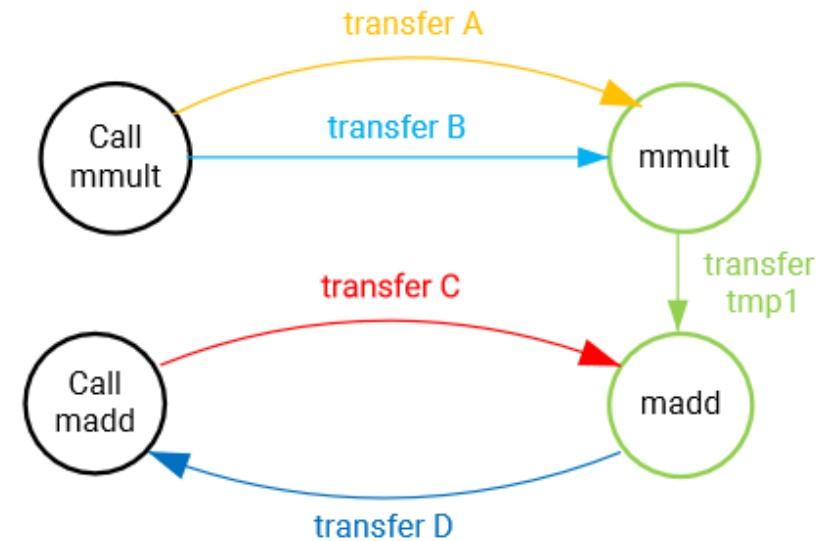
Timeline for Function Calls

```
bool mmultadd_test(float *A, float *B, float *C, float *Ds, float *D)
{
    float tmp1[A_NROWS * A_NCOLS], tmp2[A_NROWS * A_NCOLS];
    for (int I = 0; ii < NUM_TESTS; i++) {
        mmultadd_init(A, B, C, Ds, D);

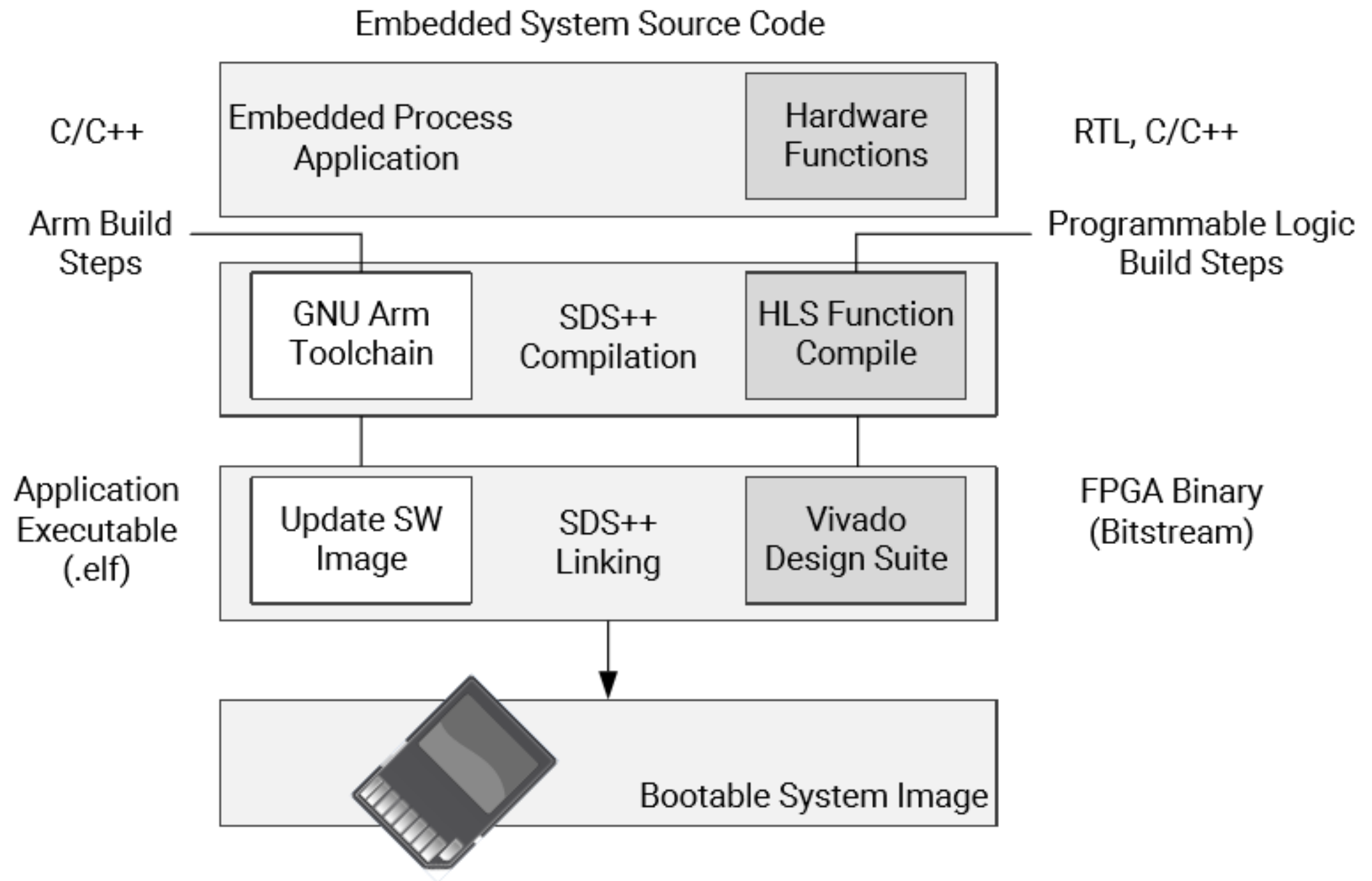
        mmult(A, B, tmp1);
        //std::cout << "tmp1[0] = " << tmp1[0] << std::endl;
        madd(tmp1, C, D);

        mmult_golden(A, B, tmp2);
        madd_golden(tmp2, C, Ds);

        if (!mmult_result_check(D, Ds))
            return false;
    }
    return true;
}
```



SDSoC Build Process



Build Compilation Process

The compilation process includes the following tasks:

- Analyzing the code and running a compilation for the main application on the Arm core, as well as a separate compilation for each of the hardware accelerators.
- Compiling the application code through standard GNU Arm compilation tools with an object (.o) file produced as final output.
- Running the hardware accelerated functions through the HLS tool to start the process of custom hardware creation with an object (.o) file as output.

Build Linking Process

After compilation, the linking process includes the following tasks:

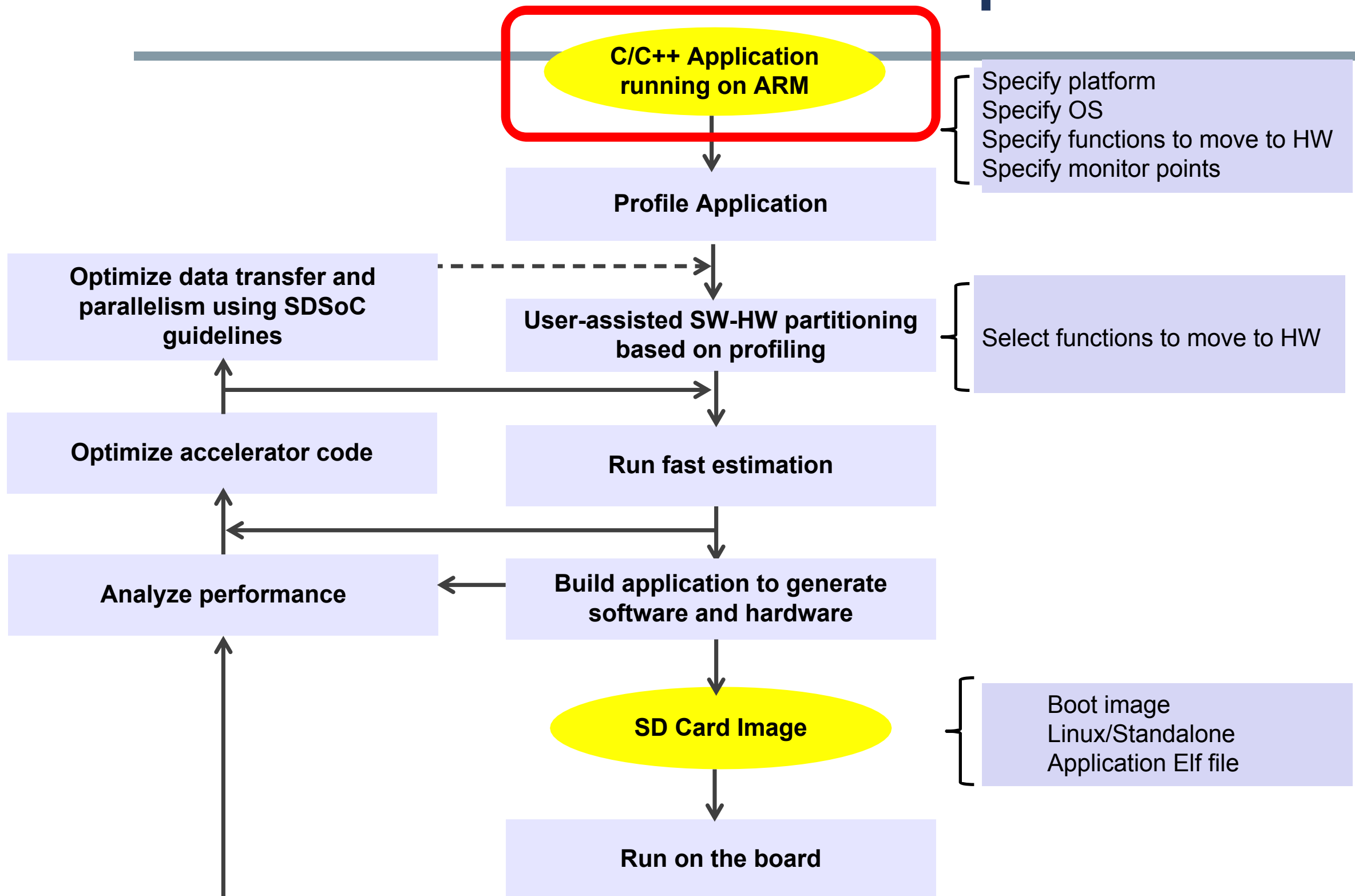
- Analyzing the data movement through the design and modifying the hardware platform to accept the accelerators.
- Implementing the hardware accelerators into the programmable logic (PL) region using the Vivado Design Suite to run synthesis and implementation, and generate the bitstream for the device.
- Updating the software images with hardware access APIs to call the hardware functions from the embedded processor application.
- Producing an integrated SD card image that can boot the board with the application in an Executable and Linkable Format (ELF) file.

Programming for SDSoC

- Create a SDSoC Project
- SDSoC Execution and Building
- 👉 Programming for SDSoC

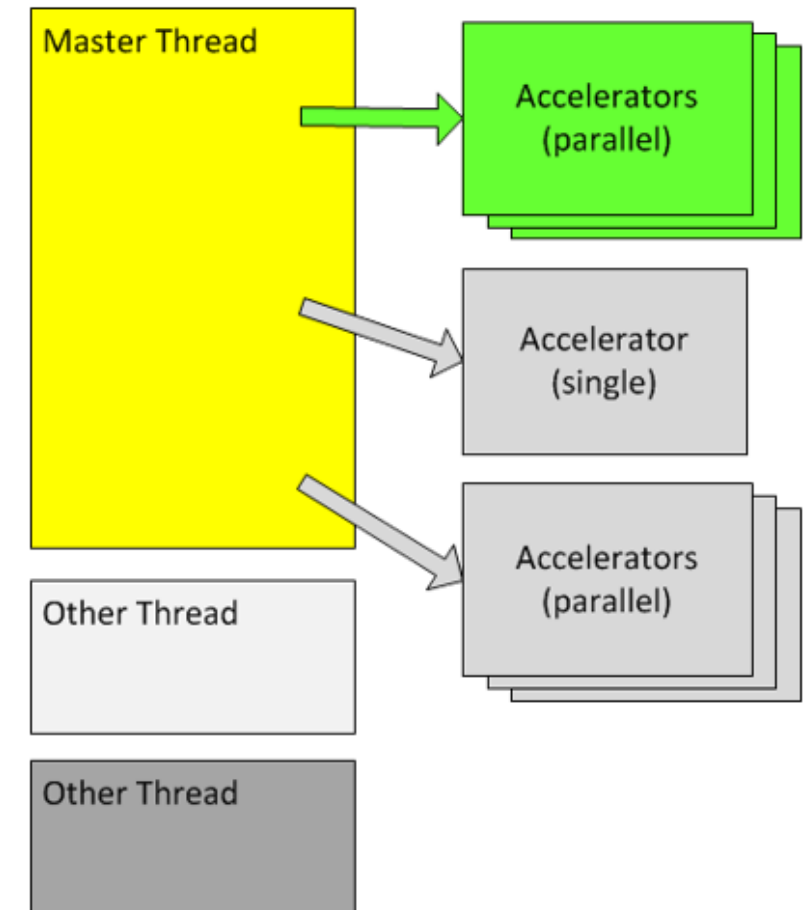


Development Flow



General C/C++ Guidelines

- Hardware functions can execute concurrently under the control of a single master thread
 - A program can have multiple threads and processes
 - Concurrently means that there are multiple copies of the accelerator (parallelism)
- No support for exception handling in hardware functions
 - Cannot *break* from accelerator
 - Global variables are synthesizable but it is not recommended to use them heavily
 - Potential issues with concurrency



Basic Compile

- Ensure the project compiles correctly as a *software only* implementation
 - Deselect all functions from porting to hardware
 - Compile the project under SDDebug configuration
 - Compiler report located in the console and in .jou and .log files
 - If there are any C/C++ issues, resolve them
 - SDSoC tool/HLS pragma have no effect on the software, only on implementation
- Proceed to the next step when the code compiles correctly



Verify Tool Operation for Dummy Hardware Function

➤ Add and mark a "known good" function for acceleration

– Add and call a "known good" function to the software design

- Available from the *samples* directory
- Can use customer-built function

– Mark the simple function for acceleration

– Compile for the Estimate Performance

– If the build fails

- Review the console output for information

– Check for any C syntax issues that are valid for C but invalid for SDSoC

```
int simple_function(uint32_t *  
ptr, uint32_t n) {  
    int index = 0;  
    while (n--) { * (ptr + index++)  
= index;    }  
    return 0;  
}
```

➤ If it compiles, then the source code is syntactically acceptable for the tools "under the hood" and you can proceed



Verify Synthesizability of Individual Functions

➤ Mark functions for acceleration one at a time

- Mark a single high-level function for acceleration
- Compile for the Estimate Performance and review results
 - Review the console output/report
 - Any issues are most likely due to the accelerator code
 - Comment out any pragma for this accelerator and rebuild for Estimate
- Selectively add back in pragmas
 - Improper pragma syntax can be difficult to debug and will cause issues

➤ When done with each accelerated function

- Remove validated function from hardware
- Repeat process for all other accelerated functions
- Alternately, running the integration step may also be a good choice depending on the situation



Integration – Putting All Together

➤ Mark multiple functions for hardware acceleration

- Mark the desired functions for acceleration
 - Use Estimate Performance with Debug configuration initially
 - Use Release configuration once everything is in place
- If it does not build
 - Is it exceeding capacity of PL?
 - Are there port/bandwidth conflicts?
 - Review console output/logs
 - Incompatible #pragma being defined

